Cera Functional Language - 1.1

Isaac Honeyman

July 22, 2026

Abstract

This document outlines the architectural pipeline, implementation phases, and execution mechanics of the Cera custom functional programming language compiler and runtime system.

Contents

1 Core Design Ideas

1.1 Philosophy & Paradigm

- **True Functionality:** The language adheres strictly to pure functional paradigms, enforcing zero side effects (excluding isolated I/O) and strictly immutable state.
- **First-Class Functions:** Functions are treated as fundamental data types, supporting higher-order programming, closures, and localized lambda expressions.
- **Evaluation Strategy:** The language enforces strict, call-by-value evaluation. However, deferred execution and lazy infinite structures can be elegantly synthesized via `unit` thunks (e.g., `unit -> g`).
- **Simplistic, Streamlined Design:** The language minimizes syntactic bloat. Rather than offering a dozen slightly varying keywords to solve a problem, Cera focuses on a highly orthogonal set of fundamental operations that can be composed to solve any complex task.

1.2 Architecture & Execution

- **Tech Stack:** The bytecode compiler, including the lexical, syntactic, and semantic pipelines, is authored in C# to leverage modern pattern-matching and immutable records. The runtime Virtual Machine is written in C for deterministic memory control and high-performance execution.
- **Type System:** Cera utilizes a Hindley-Milner (HM) type inference engine. Global function signatures require explicit typing, establishing a rigid structural boundary, while local variable bindings benefit from seamless HM type inference.
- **Memory Model:** Heap allocations for recursive Algebraic Data Types and closures are managed via a deterministic Reference Counting system.
- **Backwards Compatibility:** The AST and bytecode serialization maintain strict versioning rules to ensure legacy Cera code compiles flawlessly on newer runtime iterations.
- **Build System:** The compilation, testing, and deployment pipeline is orchestrated via a unified Bash scripting ecosystem.
-
- **File Extensions:** Human-readable source code is strictly authored in `.cera` files. Upon successful semantic and syntactic analysis, the compiler synthesizes these files into deterministic binary object executables denoted by the `.cerabc` (Cera ByteCode) extension.

1.3 Lexical Conventions

- **OOP Style Syntax:** While mathematically pure, the surface syntax deliberately mirrors C-family/OOP languages (C#, Java) to maximize readability and reduce the barrier to entry for modern developers.
- **Variables:** Identifiers utilize `camelCase`. Explicit type annotations are encouraged for documentation but can be omitted when contextually obvious to the HM inference engine.
- **Functions:** Function identifiers enforce `camelCase` and strictly require an explicit type signature. This rule applies uniformly across user-defined and compiler-intrinsic functions.
- **Types:** Built-in primitives and user-defined Algebraic Data Types strictly use `lowercase` identifiers, whereas their concrete data constructors require `PascalCase`.

2 Language Syntax

2.1 Types

The language base type set will be extremely minimal, all using lowercase names, with the following types and type 'extensions':

- **int:** 64-bit signed integer (2's complement)
- **float:** 64-bit IEEE 754 floating point number
- **bool:** 1-bit boolean value (true or false)
- **char:** 32-bit Unicode character
- **unit:** A type representing the absence of a meaningful mathematical value, used for side effects, and delayed evaluation. It has exactly one valid value, written as `()`.
- **list:** linked list of values of a single type (OCaml-style)
- **arr:** fixed-length contiguous array, assignable once upon creation, addressable via the built in get keyword.
- **tup:** fixed-length tuple of values of potentially different types, note this is not a keyword unlike other extensions, and type definition must be wrapped in brackets to prevent ambiguity.

Note that in the following snippets, we can chain together extensions to create more complex types, as well as the syntax sugar for a char list.

Cera Language

```
def foo() : unit = {
    var x: int = 5;
    var y: int list = [1, 2, 3, 4];
    var z: char list list = [['a', 'b'], ['c', 'd']];
    var s: char list = "hello world";
    var w: bool arr = arr[true, false, true];
    var u: (int * int * int) = (1, 2, 3);
    var f: int -> int -> int =
      func (x: int, y: int) : int = x + y;
    ();
}
```

2.2 Variable Declarations and Scoping

Variables in Cera are immutable bindings, explicitly or implicitly typed upon creation. Because the language enforces zero mutable state, a variable's value can never be altered after it is bound.

- **Bindings:** Variables are declared using the `var` keyword. If a type annotation is omitted, the compiler utilizes local type inference to determine the type from the evaluated expression.
- **Shadowing:** Cera supports variable shadowing within the same scope. Re-declaring a variable with an existing identifier creates a completely new memory binding that obscures the original identifier for the remainder of the scope, preserving immutability.
- **Strings:** The language does not have a distinct string primitive; syntactic sugar can be used, and strings are evaluated syntactically as a `char list`.

- **Global Constants:** Variables declared at the top level of a file belong to the global environment. To enforce deterministic memory initialization and maintain absolute functional purity prior to the `entry` execution, these top-level declarations are strictly constrained to literal values. They may be preceded by the `hidden` keyword to restrict their scope entirely to the defining file.

Data Transformation via Shadowing

```
def normalizeVector(x: float, y: float, z: float) :
(float * float * float) = {

    // 1. Calculate the magnitude
    var mag: float = (x * x) + (y * y) + (z * z);

    // 2. Safely handle zero-vectors without mutating state
    var mag: float = (mag == 0.0) ? 1.0 : mag;

    // 3. 'mag' is shadowed; we securely divide by the new 'mag'
    (x / mag, y / mag, z / mag);
}
```

Destructuring via Variable Declarations

```
var consts = (3.14, 2.72)

def calculateExtents(points: (float * float) list) : unit = {
    // 1. Unpacking a constructor payload
    var Ok(data) = processPoints(points);

    // 2. Unpacking a tuple directly into local variables
    var (pi, e) = consts;
    var (minX, minY) = getMinBounds(data);
    var (maxX, maxY) = getMaxBounds(data);

    outF("X: % to %", [floatToChars(minX), floatToChars(maxX)]);
    ();
}
```

2.3 Operators and Expressions

In Cera, all operations are evaluated strictly by default. The language supports standard arithmetic, relational, and logical operators. Because Cera is a strictly typed language, operators do not implicitly cast between types (e.g., an `int` cannot be added directly to a `float` without explicit conversion). This process shall be done by built-in function calls.

- **Arithmetic:** Standard binary operators for numeric types: `+`, `-`, `*`, `/`, and `%` (modulo) (integer and float).
- **Bitwise:** Bitwise operators: `&`, `|`, `^`, `~`, `<<`, and `>>` (integer).
- **Relational:** Comparison operators returning a `bool`: `==`, `!=`, `<`, `>`, `<=`, `>=`. Structural equality is evaluated for composite types like tuples and lists (only `==` and `!=`).
- **Logical:** Boolean operators: `&&` (logical AND), `||` (logical OR), and `!` (logical NOT). These support short-circuit evaluation.

- **Concatenation and Lists:** The `::` operator is used to prepend an element to a list. The use of a `concat` function will concatenate two lists/arrays of the same type.
- **Conditional:** Use of the `?` and `:` operators for inline conditional expressions (ternary operator).

Expressions and Operators

```
def foo() : unit = {
    var a: int = 10 + 5;
    var b: float = 3.14 * 2.0;
    var isValid: bool = (a > 20) && (b < 10.0); // false
    var sameTuple: bool = (1, 2) == (1, 2); // true
    var combined: int list = concat([1, 2], (3::[4, 5]));
    var result: int = isValid ? 100 : a > 20 ? 200 : 300; // 300
    ();
}
```

2.4 Control Flow

Because Cera enforces zero mutable state, standard control flow statements are replaced by **control flow expressions**. Every conditional construct evaluates to a return value. To maintain type safety, the compiler strictly enforces that all branches within a single control flow construct evaluate to the exact same type.

- **Expression Blocks:** Standard `if/else` and `else if` blocks that evaluate to the value of their final expression. The `return` keyword is not used; the last evaluated expression in the block is returned implicitly. Note that the final expression of a block does not require a `;` and the inclusion is up to a programmers intuition.
- **Ternary Operator:** The inline `? :` operator is supported for concise, single-line conditional assignments. Chained ternary operators are right-associative.
- **Pattern Matching & Guards:** The `switch` construct replaces traditional statement-based switch blocks. It is the primary mechanism for structurally unpacking tuples, lists, and complex types. Branches are defined using the `->` operator. Additionally, cases may include an optional `if (condition)` guard clause immediately after the pattern. This leverages a test-then-bind architecture to evaluate boolean constraints before fully committing to the execution branch.

Pattern Matching with Guards

```
def evaluateFlags(flags: char list list, verbose: bool)
    : bool = {
    switch (flags) {
        // Base case: stream is empty
        [] -> verbose,

        // Destructure and apply a boolean guard clause
        (h::tl) if (h == "v") -> {
            evaluateFlags(tl, true);
        },

        // Fallback for unknown flags
        (_::tl) -> evaluateFlags(tl, verbose)
    }
```

```
}
```

2.5 Functions

Functions in Cera are first-class citizens. Because the language enforces zero mutable state, there are no traditional looping constructs (e.g., `for` or `while` loops); all iteration is achieved via recursion.

- **Declarations:** Named functions are declared using the `def` keyword. All parameters and the return type must be explicitly annotated. The body of the function is an expression block, and the final evaluated expression is implicitly returned.
- **Higher-Order Functions:** Functions can be passed as arguments to other functions and returned as values, utilizing the arrow typing syntax (e.g., `int -> int`).
- **Recursion:** Functions in Cera are recursive by default, allowing a function to call itself within its own body without additional keywords.
- **Anonymous Functions (Lambdas):** Lambda functions are defined using the `func` keyword, followed by explicitly typed parameters, a strictly required return type annotation, and either an expression block or a single inline expression.
- **Generic Functions:** Cera supports generic functions by appending type parameters in angle brackets directly after the function name (e.g., `def name<g1, g2>`). These generic types can be used throughout the function signature and body.
- **Scoping and Shadowing:** All named functions (declared via `def`) must reside in the global scope and possess strictly unique identifiers. Function shadowing is not permitted, and named inner functions are forbidden to maintain a flat AST. Localized closures must be handled via anonymous lambda functions.
- **Function Application:** Cera does not support automatic partial function application (currying). A function call must supply the exact number of arguments specified in its signature. Developers requiring partially applied functions must explicitly construct them using lambdas.
- **Modifiers:** Named functions declarations can be preceded by either or both the `hidden` and `inline` keywords, which restrict the scope of the function to the current file, and optimise the bytecode emission of the function respectively.

Higher-Order Data Pipelines & Modifiers

```
def sumEvenSquares(lst: int list) : int = {
    // 1. Filter out the odd numbers
    var evens = filter(lst, func (x: int): bool = x % 2 == 0);

    // 2. Map the remaining numbers to their squares
    var squares = map(evens, func (x: int): int = x * x);

    // 3. Fold the list to accumulate the final sum
    foldLeft(squares, func (acc: int, x: int): int = acc + x, 0);
}

hidden inline def square(x: int) : int = {
    x * x;
}
```

```
def sumOfSquares(a: int, b: int) : int = {
    // The compiler intercepts the calls, generating:
    // (a * a) + (b * b)
    square(a) + square(b);
}
```

2.6 Custom Types (Algebraic Data Types)

Cera supports the creation of custom Algebraic Data Types (ADTs). To align with the built-in primitive types, user-defined type identifiers are strictly lowercase, while their data constructors must be capitalized (PascalCase). This guarantees the lexer can instantly distinguish between a type signature and a concrete data constructor.

- **Sum Types (Variants):** Defined using the `type` keyword. Constructors are separated by the `|` (OR) operator. If a constructor carries a data payload, the `:` operator is used to specify the underlying type.
- **Recursive Types:** Types can reference themselves within their own definitions, allowing for recursive data structures like trees and streams.
- **Generic Types:** Type parameters are declared using angle brackets (e.g., `<g>`) and passed consistently to recursive calls.
- **Encapsulation:** Type declarations may be preceded by the `hidden` keyword. This strictly isolates the type identifier and all of its concrete data constructors to the defining file, preventing external modules from pattern-matching against internal data structures.

Recursive Immutable Data Structures

```
// A generic Binary Tree definition
type tree<g> = Leaf | Node : (tree<g> * g * tree<g>);

def insertInteger(t: tree<int>, val: int) : tree<int> = {
    switch (t) {
        Leaf -> Node(Leaf, val, Leaf),
        Node(left, v, right) ->
            if (val < v) {
                Node(insertInteger(left, val), v, right) }
            else if (val > v) {
                Node(left, v, insertInteger(right, val)) }
            else { t } // return the unmodified tree
    }
}
```

2.7 Error Handling

To maintain absolute mathematical purity and predictable control flow, Cera completely omits `try/catch` exception blocks and the concept of `null` references. Instead, functions that can fail or return no value must make that failure explicit in their type signature using built-in Algebraic Data Types.

- **The Option Type:** Used when a function might legitimately return nothing. It is defined as `type option<g> = None | Some: g`.

- **The Result Type:** Used when a function can fail and needs to return an error message or code. It is defined as `type result<v, e> = Ok: v | Error: e`.

These types force the programmer to handle both the success and failure states using `switch` expressions, guaranteeing that unhandled exceptions cannot crash the VM.

Error Handling via ADTs

```
def divide(a: float, b: float): result<float, string> = {
    if (b == 0.0) {
        Error("Division by zero");
    } else {
        Ok(a / b);
    }
}
```

2.8 Deferred Evaluation

Cera evaluates all expressions strictly (call-by-value) by default. However, to support infinite data structures and deferred execution, one may use unit thunks.

Infinite Fibonacci Stream

```
type stream<g> = Nil | Cons : (g * (unit -> stream<g>));

def fibonacci(a: int, b: int) : stream<int> = {
    // Defer the recursive calculation behind a unit lambda
    var nextElement =
        func (u: unit) : stream<int> = fibonacci(b, a + b);

    Cons(a, nextElement);
}

def fibStream() : stream<int> = {
    fibonacci(0, 1);
}
```

2.9 Program Entry and I/O

Cera enforces zero mutable state and forbids user-defined side effects. However, a program must interact with the host system to be useful. This is achieved through controlled, built-in impure functions (intrinsics) and a strictly defined execution entry point.

- **Impure Intrinsics:** I/O operations (such as reading files or writing to standard output) are provided exclusively as compiler built-ins. Users cannot author their own impure functions from scratch within Cera.
- **The Entry Point:** Program execution begins strictly at the `entry` function. This function serves as the bridge between the host operating system and the pure functional runtime. It takes an array of command-line arguments and must evaluate to an integer representing the system exit code (where 0 indicates success).

Program Entry

```
def entry(args: char list arr): int = {
    // Built-in impure function call
```

```
    out("hello world");
    var status: int = 0; // Evaluates to the exit code
    status;
}
```

2.10 Miscellaneous Syntax

- **Comments:** Cera supports single-line comments using the standard `//` syntax. The lexer ignores all text from the `//` token to the end of the line. Block comments can be implemented using `/*` to start and `*/` to end, allowing for multi-line comments that can be nested.
- **Multifile Programs:** Cera takes a C-style approach to simple multifile programs. To use another file, a Cera file must begin with the syntax `import "FileName";`. These are then recursively dealt with (keeping track of imported files to prevent cyclic loops) and are treated as essentially one large file. Note that due to this, types and function identifiers must be unique across all files.

2.11 Summary

To assist in the lexical analysis phase, the following is an exhaustive list of reserved keywords in the Cera language. These identifiers are strictly reserved by the grammar and cannot be used as variable, function, or custom type names.

- **Declaration & Binding:** `var`, `def`, `func`, `type`, `hidden`, `inline`
- **Control Flow:** `if`, `else`, `switch`, `import`
- **Primitive Types:** `int`, `float`, `bool`, `char`, `list`, `arr`, `unit`
- **Boolean Literals:** `true`, `false`

Note: Identifiers such as `concat`, `out`, and `entry` are considered built-in intrinsic functions rather than reserved lexical keywords. Next is a list of built-in ADTs:

- **Option:** `type option<g> = None | Some: g`
- **Result:** `type result<v, e> = Ok: v | Error: e`

Finally, the list of built-in functions:

- **Get:** `get<g>(target: g arr, index: int) : option<g>` (0 based array index).
- **Arr Length:** `arrLength<g>(array: g arr) : int`
- **Concat:** `concat<g>(left: g list, right: g list) : g list`
- **Arr Concat:** `arrConcat<g>(left: g arr, right: g arr) : g arr`
- **Out:** `out(output: char list) : unit`
- **In:** `in() : char list`
- **Read:** `read(path: char list) : result<char list, char list>`
- **Write:** `write(path: char list, content: char list) : result<unit, char list>`
- **Append:** `append(path: char list, content: char list) : result<unit, char list>`

- **Int To Float:** `intToFloat(x: int) : float`
- **Float To Int:** `floatToInt(x : float) : int` (truncates)
- **Char To Int:** `charToInt(c : char) : int`
- **Int To Char:** `intToChar(x : int) : char`
- **Int To Chars:** `intToChars(x : int) : char list`
- **Float To Chars:** `floatToChars(x : float) : char list`
- **Bool To Chars:** `boolToChars(x : bool) : char list`
- **Chars To Int:** `charsToInt(str : char list) : option<int>`
- **Chars To Float:** `charsToFloat(str : char list) : option<float>`
- **Arr To List:** `arrToList<g>(array: g arr) : g list`
- **List To Arr:** `listToArr<g>(lst: g list) : g arr`
- **Random Float:** `rand() : float` (random float from 0 to 1 (inclusive)).
- **Random Int:** `randInt() : int` (random 64 bit integer (inclusive)).
- **Square Root:** `sqrt(x: float) : float`
- **Time:** `time() : int` (Wall-clock time in milliseconds from 01/01/1970 UTC).
- **Local Time:** `timeLocal() : int` (Wall-clock time in milliseconds from 01/01/1970 local time).
- **Uptime:** `uptime() : int` (Monotonic time in milliseconds since the host system booted).
- **Read Binary:** `readBin(path: char list) : result<int arr, char list>` (reads raw bytes into an integer array).
- **Write Binary:** `writeBin(path: char list, data: int arr) : result<unit, char list>` (writes an integer array as raw bytes).
- **Natural Logarithm:** `ln(x: float) : float` (computes n-dimensional natural logarithm $\ln(x)$ via FPU).
- **Floating-Point Power:** `pow(base: float, exp: float) : float` (computes floating-point exponentiation base^{exp}).

3 Lexical Analysis (Tokenisation)

The lexical analysis phase operates as a Deterministic Finite Automaton (DFA), consuming the raw UTF-8 character stream of the source file and segmenting it into a linear sequence of logical `Token` structures. This process strips away non-semantic formatting, preparing the data for syntax analysis.

3.1 The Token Structure

Each emitted token encapsulates three critical pieces of data to ensure both accurate parsing and robust error reporting:

- **Tag (Type):** An enumerated value classifying the token (e.g., `IDENTIFIER`, `INT_LIT`, `DEF_KW`, `PLUS_OP`).

- **Lexeme:** The concrete string of characters that formed the token.
- **Location Metadata:** The line and column index marking the token's origin, strictly utilized for traceback generation during semantic or syntactic failures.

3.2 Lexical Categories and Regular Expressions

The DFA categorizes lexemes based on the following rules and regular expressions. The scanner employs the "maximal munch" principle, always matching the longest possible valid lexeme (e.g., reading `<=` as a single relational operator rather than a `<` followed by an `=`).

- **Identifiers:** Variables and functions map to `[a-z][a-zA-Z0-9_]*`. Custom type constructors must begin with a capital letter, mapping to `[A-Z][a-zA-Z0-9_]*`.
- **Keywords:** Reserved language keywords (e.g., `var`, `def`, `switch`) are initially scanned as standard identifiers. Before emission, they are checked against a static hash map and re-tagged with their specific keyword enumeration to achieve $O(1)$ lookup times.
- **Literals:**
  - *Integer:* `[0-9]+`
  - *Float:* `[0-9]+.[0-9]+`
  - *Character:* `'[^']'`
- **Operators and Punctuation:** Single-character punctuation (e.g., `+`, `*`, `{`, `;`) transitions immediately to an accept state. Potential multi-character operators (e.g., `==`, `->`, `::`) require a single character of lookahead to determine the correct transition.

The following is a full list of tokens used within the lexer:

- **Declaration:** `Var`, `Def`, `Func`, `Type`
- **Control Flow:** `If`, `Else`, `Switch`, `Import`
- **Primitive Types:** `Int`, `Float`, `Bool`, `Char`, `List`, `Arr`, `Unit`, `WildCard`
- **Literals:** `True`, `False`, `Identifier`, `Constructor`, `IntLiteral`, `FloatLiteral`, `CharLiteral`, `StringLiteral`
- **Flow:** `LBracket`, `RBracket`, `LBRace`, `RBrace`, `LPar`, `RPar`
- **Structural:** `Equal`, `SemiColon`, `Comma`
- **Operators:** `Plus`, `Minus`, `Star`, `Slash`, `Mod`, `BitAnd`, `BitXor`, `BitNot`, `LShift`, `RShift`, `EqualEqual`, `NotEqual`, `Lesser`, `LesserEqual`, `Greater` `GreaterEqual`, `And`, `Or`, `Not`, `ColonColon` `Question`, `Colon`, `Pipe`, `Arrow`,
- **Special:** `None`

Note: The `None` token type is simply used for errors, and represents the absence of a token.

3.3 Whitespace and Comment Handling

Whitespace characters (spaces, tabs, carriage returns, and newlines) serve strictly as lexeme delimiters. When the DFA encounters whitespace, it forces the completion and emission of the currently accumulating token. The whitespace itself is then immediately consumed and discarded; no token is generated.

Similarly, single-line (`//`) and block (`/* ... */`) comments transition the scanner into an absorption state, aggressively consuming characters until the termination sequence is reached, emitting nothing to the parser stream.

3.4 Import Handling & Module Encapsulation

Cera implements a robust, Directed Acyclic Graph (DAG) module system. The `import "FileName.cera";` syntax is parsed directly into the Abstract Syntax Tree.

To guarantee $O(1)$ compilation caching and prevent cyclic dependency loops, the compiler orchestrates imports topologically. Each file is lexed and parsed exactly once. Furthermore, Cera strictly enforces module encapsulation via two-tier environments:

- **Standard Imports:** Merges the public API of the target file into both the local execution scope and the public export scope, allowing transitive dependencies.
- **Hidden Imports:** Using the syntax `hidden import "FileName.cera";`, the compiler merges the target's public API strictly into the local execution scope. The dependency is trapped within the importing file and cannot leak to external modules, enforcing rigid data hiding.

4 Syntax Analysis (Parsing)

4.1 Notation Guide

The syntax is formally specified using Extended Backus-Naur Form (EBNF). To avoid ambiguity between the meta-syntax and the language's own lexical tokens, the following typographic conventions are strictly applied:

- ⟨Rule⟩: Angle brackets denote a **non-terminal** production rule.
- **keyword**: Bold, monospace text denotes a **terminal** symbol (a concrete lexical token, keyword, or required punctuation like (or }).
- ::=: The definition operator, read as "is defined as".
- |: The alternation operator, denoting a choice between derivations ("or").
- [...]: Square brackets enclose an **optional** component (zero or one occurrences).
- (...): Standard mathematical parentheses denote **logical grouping** of meta-symbols. They are *not* literal characters in the source code.
- x^*: The Kleene star suffix indicates **zero or more** consecutive repetitions of x.
- x^+: The Kleene plus suffix indicates **one or more** consecutive repetitions of x.

4.2 Context-Free Grammar (CFG)

The formal CFG for Cera is defined in Backus-Naur Form (BNF). To satisfy structural requirements for unambiguous derivations and Top-Down parsing without left recursion, the grammar stratifies expressions and types. The grammar enforces strict functional purity by restricting `var` bindings strictly to local expression blocks.

Are top level fundamental definitions:

⟨File⟩ ::= (⟨ImportDecl⟩ | ⟨FuncDecl⟩ | ⟨TypeDecl⟩ | ⟨TopVarDecl⟩)*
⟨ImportDecl⟩ ::= [**hidden**] **import str_lit;**
⟨TopVarDecl⟩ ::= [**hidden**] **var id** [**:** ⟨Type⟩] = ⟨Literal⟩**;**
⟨FuncDecl⟩ ::= [**hidden**] [**inline**] **def id** [⟨GenericDecl⟩] **(**[⟨Param⟩ (**,** ⟨Param⟩)*]**) :** ⟨Type⟩ = ⟨ExprBlock⟩
⟨TypeDecl⟩ ::= [**hidden**] **type id** [⟨GenericDecl⟩] **=** ⟨ConDecl⟩ (| ⟨ConDecl⟩)*;
⟨GenericDecl⟩ ::= <**id** (**,** **id**)*>
⟨Type⟩ ::= ⟨SuffixType⟩ [−> ⟨Type⟩]
⟨SuffixType⟩ ::= ⟨AtomType⟩ [**list** | **arr**]*
⟨AtomType⟩ ::= ⟨BaseType⟩ | ⟨TupleType⟩ | ⟨GenericType⟩ | **(**⟨Type⟩**)**
⟨BaseType⟩ ::= **int** | **float** | **bool** | **char** | **unit**
⟨TupleType⟩ ::= **(**⟨Type⟩ (***⟨Type⟩)+**)**
⟨GenericType⟩ ::= **id**<⟨Type⟩ (**,**⟨Type⟩)*>
⟨ConDecl⟩ ::= **con** [**:** ⟨Type⟩]
⟨ExprBlock⟩ ::= **{**⟨Stmt⟩* ⟨Expr⟩[**;**]**}**
⟨Stmt⟩ ::= ⟨VarDecl⟩ | ⟨Expr⟩**;**
⟨Param⟩ ::= **id :** ⟨Type⟩

Following are the expression definitions:

⟨Expr⟩ ::= ⟨BinOpChain⟩ [**?** ⟨Expr⟩ **:** ⟨Expr⟩]
⟨BinOpChain⟩ ::= ⟨UnaryExpr⟩ (⟨BinOp⟩ ⟨UnaryExpr⟩)*
⟨UnaryExpr⟩ ::= ⟨UnOp⟩ ⟨UnaryExpr⟩ | ⟨CallExpr⟩
⟨CallExpr⟩ ::= ⟨Primary⟩ (([⟨Expr⟩(, ⟨Expr⟩)*]))*
⟨Primary⟩ ::= ⟨Literal⟩ | **id** | **(**⟨Expr⟩**)** | ⟨IfExpr⟩ | ⟨SwitchExpr⟩ | ⟨Lambda⟩
⟨BinOp⟩ ::= + | **-** | ***** | **/** | % | **&** | | | ^ | << | >> | == | ! = | < | > | <= | >= | **&&** | || | **::**
⟨UnOp⟩ ::= **!** |∼| **-**
⟨IfExpr⟩ ::= **if (**⟨Expr⟩**)** ⟨ExprBlock⟩ (**else if (**⟨Expr⟩**)** ⟨ExprBlock⟩)* [**else** ⟨ExprBlock⟩]
⟨SwitchExpr⟩ ::= **switch (**⟨Expr⟩**) {**⟨PatternMatch⟩ (**,** ⟨PatternMatch⟩)***}**
⟨PatternMatch⟩ ::= ⟨Pattern⟩ [**if (**⟨Expr⟩**)**] −> (⟨Expr⟩ | ⟨ExprBlock⟩)
⟨Pattern⟩ ::= ⟨Literal⟩ | **id** | _ | **(**⟨Pattern⟩ (**,** ⟨Pattern⟩)+**)** | [[⟨Pattern⟩ (**,** ⟨Pattern⟩)*]]
| **arr** [[⟨Pattern⟩ (**,** ⟨Pattern⟩)*]] | **(**⟨Pattern⟩ **::** ⟨Pattern⟩**)**
| **con**[**(**⟨Pattern⟩ (**,** ⟨Pattern⟩)***)**]
⟨Lambda⟩ ::= **func (**[⟨Param⟩ (**,** ⟨Param⟩)*]**) :** ⟨Type⟩ **=** (⟨Expr⟩ | ⟨ExprBlock⟩)
⟨Literal⟩ ::= **int_lit** | **float_lit** | **char_lit** | **str_lit** | **true** | **false** | **()** | ⟨ListLit⟩ | ⟨ArrLit⟩ | ⟨TupleLit⟩
⟨ListLit⟩ ::= [[⟨Expr⟩ (**,** ⟨Expr⟩)*]]
⟨ArrLit⟩ ::= **arr** [[⟨Expr⟩ (**,** ⟨Expr⟩)*]]
⟨TupleLit⟩ ::= **(**⟨Expr⟩ (**,** ⟨Expr⟩)+**)**

Note: That **id** and **con** refer to identifier and constructor tokens respectively.

4.3 Lexical Ambiguity Resolution

Because the lexical analyser adheres strictly to the "maximal munch" principle, the character sequence `>>` is always greedily tokenised as a single right-shift operator (`>>`), rather than two

distinct `>` tokens. This creates a standard structural collision in $LL(k)$ parsers when evaluating nested generic type closures (e.g., `option<stream<g>>`).

To resolve this without breaking lexical encapsulation or resorting to a "lexer hack", the ambiguity is handled entirely within the syntax analyser. When the parser is evaluating a generic type signature and expects a closing bracket `>`, but instead encounters a right-shift `>>` token, it consumes the operator, logically applies the first `>`, and modifies the token stream (or internal parser state) to inject a synthetic `>` token for the subsequent closure evaluation.

4.4 Operator Precedence and Associativity

To eliminate structural ambiguity during the parsing of complex expressions, Cera enforces a strict, formalized hierarchy of operator precedence. The parser utilizes a Top-Down Operator Precedence (Pratt Parsing) architecture, assigning binding powers to individual operator tokens.

A critical architectural departure from legacy C-family languages is Cera's handling of bitwise operators. Historically, languages like C and Java assign bitwise AND, OR, and XOR a lower precedence than relational comparison operators, leading to non-intuitive evaluations (e.g., `a & b == c` evaluating as `a & (b == c)`). Cera corrects this by enforcing that all bitwise operations bind more tightly than equality and comparison checks, prioritizing mathematical correctness.

Operators are evaluated in the following order, from highest precedence (tightest binding) to lowest precedence:

1. **Call & Grouping:** `()`
2. **Unary Operators:** `!`, `~`, `-`
3. **Multiplicative (Factor):** `*`, `/`, `%` (Left-associative)
4. **Additive (Term):** `+`, `-` (Left-associative)
5. **List Concatenation:** `::` (Right-associative)
6. **Bitwise Shift:** `<<`, `>>` (Left-associative)
7. **Bitwise AND:** `&` (Left-associative)
8. **Bitwise XOR:** `^` (Left-associative)
9. **Bitwise OR:** `|` (Left-associative)
10. **Comparison:** `<`, `>`, `<=`, `>=` (Left-associative)
11. **Equality:** `==`, `!=` (Left-associative)
12. **Logical AND:** `&&` (Left-associative, Short-circuiting)
13. **Logical OR:** `||` (Left-associative, Short-circuiting)
14. **Conditional (Ternary):** `?  :` (Right-associative)

When operators of identical precedence appear within the same expression, their associativity dictates the order of evaluation. Left-associative operators are evaluated strictly from left to right. The *only* right-associative operators in the Cera language are list concatenation (`::`) and the ternary conditional (`?  :`), which evaluate from right to left, natively supporting recursive structural evaluation.

5 Abstract Syntax Tree (AST) Implementation

The translation of the parsed token stream into a logical hierarchy is governed by a purely immutable Abstract Syntax Tree (AST). The C# implementation completely eschews traditional, deep object-oriented inheritance hierarchies in favor of data-centric structures that mirror Cera's own functional paradigms.

5.1 Node Architecture and Immutability

The AST is constructed exclusively using C# `record` types, guaranteeing that all nodes are deeply immutable upon creation. At the root of the hierarchy lies the `FileAST`, representing a single, isolated module within the compiler's Directed Acyclic Graph (DAG). This explicitly replaces legacy monolithic root nodes, guaranteeing that every file strictly maintains its own absolute path and localized import dependencies.

To rigidly categorize nodes and enforce compile-time safety without relying on behavioral inheritance, the compiler utilizes empty marker interfaces: `INodeAST`, `IExprAST`, `IStmtAST`, `ITypeAST`, and `IPatternAST`.

This architecture ensures that structural constraints (such as a `BinaryExpr` strictly requiring a left and right `IExprAST`) are enforced by the type system, while keeping the raw data representations entirely decoupled from compiler operations.

5.2 Pratt Parsing Mechanics

Expression evaluation utilizes a Top-Down Operator Precedence (Pratt) parsing architecture, orchestrated via an `InitializePrattParser()` registry. Rather than relying on deeply nested, rigid recursive descent grammar methods for every mathematical precedence level, the parser assigns semantic parsing delegates to specific token types dynamically:

- **Prefix Parsers:** Tokens that structurally initiate an expression (e.g., literals, identifiers, unary operators, or opening brackets) are mapped to a `PrefixParseFn` delegate.
- **Infix Parsers:** Tokens that operate between existing expressions (e.g., binary operators, the ternary question mark, or the function call parenthesis) are mapped to an `InfixParseFn` delegate, explicitly paired with their operational `Precedence` weighting.

During evaluation, the parser continuously polls the token stream for its assigned binding power. It greedily consumes tokens and recursively invokes their registered delegates for as long as the upcoming token's precedence strictly exceeds the current binding context, naturally producing a correctly ordered AST.

Overall we are implementing a Hybrid LL(1) & Pratt parser.

5.3 Tree Traversal via Pattern Matching

Because the AST nodes are implemented as pure data records devoid of behavioral logic, the compiler deliberately omits the traditional Object-Oriented Visitor pattern (i.e., nodes do not implement `Accept(IVisitor)` methods).

Instead, subsequent compiler phases, such as Semantic Analysis, Type Checking, and Bytecode Emission traverse the AST utilizing modern C# functional pattern matching. By utilizing `switch` expressions over the base marker interfaces (like `IExprAST`), the compiler can cleanly and exhaustively unwrap node structures. This centralizes pipeline logic within the respective compiler phases, avoids interface bloat, and aligns the compiler's codebase with the functional design philosophy of the Cera language itself.

6 Semantic Analysis

Following syntax analysis, the compiler enters the semantic phase. This stage operates over the immutable Abstract Syntax Tree (AST), performing context-sensitive validation, scope resolution, and rigorous type checking to guarantee program safety prior to Intermediate Representation (IR) lowering.

6.1 Environment and Symbol Resolution

Scope in Cera is managed via a hierarchical, purely functional `Environment` architecture. Each local block (e.g., function bodies, lambda expressions, and pattern matching branches) generates a new environment layer linked to its parent.

- **The Scope Chain:** The environment acts as a singly-linked list of hash maps. Each block creates a new environment pointing to its enclosing parent.
- **Symbol Table & Complexity:** Identifiers are mapped to immutable `Symbol` records (e.g., `VarSymbol`, `FuncSymbol`, `TypeSymbol`, `ConstructorSymbol`). Variable lookups operate at $O(1)$ time complexity within the local scope. If a symbol is not found, the traversal up the environment chain operates at $O(K)$ time, where K is the depth of the nested scope.
- **Immutability Guarantee:** Shadowing creates a completely new `VarSymbol` in the localized $O(1)$ hash map rather than mutating the parent environment. This guarantees the language's strict immutability constraint is enforced directly at the compiler architecture level.
- **File-Level Isolation & DAG Traversal:** The monolithic global environment is strictly reserved for native C-intrinsics. User-defined files operate within isolated `LocalEnv` and `ExportEnv` layers. The semantic analyzer resolves module boundaries by performing a post-order topological traversal of the parsed ASTs, dynamically composing symbol tables to guarantee dependencies are type-checked before their importers.

6.2 Hindley-Milner Type Inference (Algorithm W)

Cera employs a robust Hindley-Milner (HM) type inference engine based on Damas and Milner's Algorithm W. While function signatures require explicit type annotations, local variable bindings (`var`) and intermediate expressions rely on HM unification to statically deduce their types.

- **Type Variables:** Unknown types are initially assigned a fresh, sequentially generated `TypeVar` (e.g., T_1, T_2). The algorithm traverses the AST bottom-up, generating type equations and solving them dynamically via unification.
- **Structural Unification:** The `Unify` function mathematically equates expected and actual types. When unifying composite types, the algorithm recursively unpacks the structures. For example, unifying `(int * ` T_1`)` with `(`T_2 `* float)` successfully resolves T_1 to `float` and T_2 to `int` through parallel structural traversal.
- **Recursive Protection:** This is a crucial safety mechanism. If a user writes an unbounded recursive function, the unification engine might attempt to equate a type variable T_1 with a function signature containing itself (e.g., $T_1 \rightarrow$ `int`). The explicit "occurs check" halts this process, preventing the compiler from falling into an infinite loop attempting to construct an infinitely sized type graph.
- **Arity and Unit Matching:** Zero-arity functions mathematically evaluate as taking a single `unit` parameter. The syntax `()` evaluates to the structural type `unit`, ensuring

complete parity between explicit thunks (`func(u: unit)`) and standard zero-argument functions.

6.3 Generic Instantiation and Polymorphism

Cera supports parametric polymorphism for functions and ADTs. To prevent generic type collisions during recursive function calls or nested data structures, the semantic analyzer utilizes a strict instantiation pipeline.

- **Polytypes vs. Monotypes:** When an Algebraic Data Type (ADT) or generic function is registered in the global environment, its signature is stored as a "polytype" containing unbound generic identifiers.
- **Preventing Variable Capture:** If the compiler unified these polytypes directly during a recursive call, it would permanently bind the global generic parameter to a specific local type, corrupting all future calls.
- **Monomorphization via Substitution:** To solve this, the `Replace` method intercepts the call. It generates a localized mapping of fresh type variables and exhaustively replaces the global parameters across the entire AST node structure, including deep traversals into `TupleType`, `ListType`, `ArrType`, and `FuncType`. This converts the generic polytype into a localized "monotype" safe for HM unification.

6.4 Operator and Expression Validation

To guarantee runtime safety and eliminate algorithmic ambiguity, semantic analysis rigidly enforces operator constraints before unification.

- **Mathematical Operations:** Operators such as `+`, `-`, `*`, and `/` forcefully constrain their operands to numeric base types (`int` or `float`). Ambiguous generic operands defaulting to these operators are strictly bound to `int`.
- **Relational Operations:** Operators such as `<` and `>=` execute numeric validation on their operands but strictly return a unified `bool` type.
- **Ad-Hoc Polymorphism Resolution:** In purely inferred functional systems, overloaded operators cause severe ambiguity when evaluating unannotated lambdas. Because Cera's grammar strictly mandates explicit type hints for all lambda and function parameters, types are statically bound before operators are evaluated. This entirely sidesteps a classic functional compiler dilemma without relying on external structures like Type Classes.

7 Bytecode Emission and Execution Architecture

The final phase of the compiler frontend translates the validated Abstract Syntax Tree (AST) into a linear sequence of machine-readable bytecodes. This Intermediate Representation (IR) is explicitly designed for a custom Stack-Environment-Control (SEC) virtual machine architecture, written in C to ensure deterministic memory alignment and rapid execution.

7.1 Virtual Machine Target Architecture

Cera's target architecture is a strict stack machine. Variables and operands are never bound to explicit hardware registers; instead, all mathematical ALU operations, memory allocations, and control flow branching implicitly operate on the top of the operand stack.

- **Instruction Format:** The Instruction Set Architecture (ISA) utilizes a variable-length format. The base opcode is always exactly 1 byte. Depending on the operation, it is

followed by 0, 1, or 2 bytes of immediate operand data (e.g., local variable indices, jump offsets, or constant pool addresses).

- **Little-Endian Encoding:** To maintain compatibility with modern x86/ARM hardware targets, all multi-byte operands within the bytecode stream (such as the 16-bit addresses for `LOAD_FUNCTION_LONG`) are encoded using Little-Endian byte order.

7.2 Operational Limitations and Capacity

To maximize execution speed and minimize cache misses during the virtual machine's fetch-decode loop, the compiler strictly enforces 8-bit and 16-bit limits on program structures. Violating these limits during emission results in a fatal compile-time error.

- **8-Bit Constraints (Maximum 255):**
  - **Arguments & Arity:** A function may accept a maximum of 255 arguments.
  - **Local Variables:** A single lexical scope (stack frame) may contain up to 255 local variable bindings.
  - **Closures:** A lambda function may capture a maximum of 255 upvalues from its enclosing environments.
  - **Destructuring:** Tuples and Arrays may contain a maximum of 255 elements when unpacked via pattern matching.
  - **ADT Constructors:** The entire module may define a maximum of 255 unique custom Algebraic Data Type constructors (tags `0x00` to `0xFF`), with the first 16 tags reserved for intrinsic compiler structures like `Option` and `Result`.
- **16-Bit Constraints (Maximum 65,535):** To accommodate larger codebases without sacrificing the performance of single-byte instructions, the compiler utilizes extended `_LONG` opcodes for structures that commonly exceed 255 elements.
  - **Global Functions:** Indexed up to 65,535 using `LOAD_FUNCTION_LONG`.
  - **Constant Pool:** Up to 65,535 distinct strings, large numbers, and floats per function using `LOAD_CONST_LONG`.
  - **Jump Offsets:** Control flow branching (if/else, switches) can skip up to 65,535 bytes of intermediate code.
  - **Array Literals:** Compile-time arrays may allocate up to 65,535 elements via `ALLOC_ARRAY_LONG`.

7.3 Tail Call Optimization (TCO)

Because Cera is a pure functional language lacking iterative looping constructs (e.g., `while` or `for`), all iteration must be achieved through recursion. To prevent Stack Overflow exceptions and catastrophic memory exhaustion, the emitter rigidly enforces Tail Call Optimization.

During AST traversal, a boolean `isTail` flag tracks the evaluation context. This flag is preserved through control flow boundaries—such as the final expression of an `ExprBlock`, or the terminal branches of an `IfExpr` and `SwitchExpr`.

When a `CallExpr` is evaluated in a tail position, the compiler emits the `TAIL_CALL` (`0x62`) instruction instead of a standard `CALL` (`0x60`). This instructs the virtual machine to securely tear down the current execution frame and overwrite it with the incoming callee's arguments, maintaining an $O(1)$ spatial complexity for unbounded recursive loops.

7.4 Pattern Matching Compilation

Compiling the `switch` expression requires complex stack discipline to handle sequential fallthrough without leaking memory. Cera employs a "Test-then-Bind" architecture:

1. **Target Duplication:** The target expression is pushed to the stack, immediately followed by the `DUP` (`0x02`) opcode. This allows individual cases to test the structural shape of the target without permanently consuming the pointer.
2. **Structural Testing:** The pattern's outer shape is evaluated via type-specific testing opcodes (e.g., `MATCH_TAG` for ADTs, `IS_LIST_EMPTY` for `Nil`, or `MATCH_ARRAY_LENGTH`). If the test fails, a `JUMP_IF_FALSE` instruction routes execution to the next case.
3. **Memory Unpacking (Binding):** If the test succeeds, the original duplicated target must be consumed. Unpacking opcodes (e.g., `UNPACK_TUPLE`, `UNPACK_CON`, `UNPACK_LIST`) extract the heap-allocated payload elements directly onto the operand stack, where they are inherently bound to the compiler's local variable tracking for the duration of the case body.
4. **Exhaustiveness & Soundness:** If all pattern cases are exhausted and execution falls through, the compiler emits a terminal `MATCH_FAIL` (`0x87`) instruction. This deliberately halts the virtual machine, preventing silent failures and guaranteeing strict evaluation soundness.

7.5 Executable Object Format (`.cerabc`)

The C# compiler does not execute code; it serializes the synthesized `Module` into a dense, standalone binary executable format. Files using this format carry the `.cerabc` (Cera ByteCode) extension. The target C virtual machine consumes this file during initialization, dynamically allocating operational memory prior to execution.

The binary file is structured sequentially into a rigid hierarchy:

1. File Header

- **Magic Number (4 bytes):** The ASCII characters `C E R A`.
- **Version Data (4 bytes):** A 32-bit integer indicating the compiler version (e.g., `1`).
- **Entry Point Index (4 bytes):** A 32-bit integer pointing to the global index of the `entry` function. If the module is a library, this evaluates to `-1`.
- **Function Count (4 bytes):** A 32-bit integer declaring the total number of functions in the module.

2. Function Blocks

For each compiled function, the file encodes a self-contained execution block:

- **Function Metadata:** Includes the Global Index (4 bytes) and the Parameter Arity (1 byte).
- **Constant Pool:** Initiated by a 16-bit Unsigned Integer (`ushort`) declaring the count of constants. Each constant is prefixed by a 1-byte `ValueTag`, dictating its subsequent payload size to the C parser:
  - `Int` & `Float`: 8-byte payload (64-bit IEEE 754 / Signed Int).

  - `Char`: 4-byte payload (UTF-32 Code Point, explicitly downcasted for memory efficiency).
  - `Bool`: 1-byte payload.
  - `Unit`: 0-byte payload. The 1-byte tag represents the entire value.
  - `String`: A 4-byte (`uint32_t`) length prefix, followed strictly by a raw UTF-8 byte array to guarantee standard cross-platform C compatibility.
- **Bytecode Sequence:** Initiated by a 32-bit integer representing the byte length of the instruction stream, followed immediately by the raw execution bytes.

7.6 Instruction Set Architecture (ISA)

The Cera Virtual Machine implements a specialized Stack-Machine ISA. Operands are implicitly consumed from and pushed to the top of the operand stack. The complete instruction set is categorized logically below, mapping the 1-byte opcodes to their strict execution semantics.

Stack Operations & Constants

- `NOP (0x00)`: No operation. Used for memory alignment or patching.
- `POP (0x01)`: Discards the top value of the stack.
- `DUP (0x02)`: Duplicates the top value of the stack (critical for pattern testing).
- `LOAD_CONST (0x03)`: Consumes a 1-byte operand representing an index in the constant pool, pushing the loaded value to the stack.
- `PUSH_0 (0x04)`: Optimized instruction to push the integer 0.
- `PUSH_1 (0x05)`: Optimized instruction to push the integer 1.
- `PUSH_BYTE (0x06)`: Consumes a 1-byte immediate operand, pushing it as an integer (for values -128 to 127).
- `PUSH_TRUE (0x07)`: Optimized instruction to push boolean `true`.
- `PUSH_FALSE (0x08)`: Optimized instruction to push boolean `false`.
- `PUSH_UNIT (0x09)`: Optimized instruction to push the `unit` literal `()`.
- `PUSH_CHAR (0x0A)`: Consumes a 4-byte immediate operand (UTF-32), pushing it as a character.
- `LOAD_CONST_LONG (0x0B)`: Consumes a 2-byte operand (Little-Endian) to load a constant from a pool exceeding 255 items.

Environment & Scoping

- `LOAD_LOCAL (0x10)`: Consumes a 1-byte index, pushing the value of a local variable from the current Call Frame's window.
- `STORE_LOCAL (0x11)`: Consumes a 1-byte index, popping the top stack value and storing it in the local scope.
- `LOAD_UPVALUE (0x12)`: Consumes a 1-byte index, pushing a captured variable from the active closure's heap-allocated upvalue array.

- `LOAD_FUNCTION (0x13)`: Consumes a 1-byte global index, pushing a function pointer to the stack.
- `LOAD_FUNCTION_LONG (0x14)`: Consumes a 2-byte global index (Little-Endian), pushing a function pointer.

Arithmetic & Logic Unit (ALU)

Note: All binary ALU operations pop two values (Right, then Left), compute the result, and push it to the stack.

- **Mathematical:** `ADD (0x20)`, `SUB (0x21)`, `MUL (0x22)`, `DIV (0x23)`, `MOD (0x24)`.
- **Unary Math:** `NEGATE (0x25)` pops a number and pushes its arithmetic negation.
- **Bitwise:** `BIT_AND (0x30)`, `BIT_OR (0x31)`, `BIT_XOR (0x32)`, `SHL (0x34)`, `SHR (0x35)`.
- **Unary Bitwise:** `BIT_NOT (0x33)` pushes the one's complement of the popped integer.
- **Relational:** `EQ (0x40)`, `NEQ (0x41)`, `LT (0x42)`, `GT (0x43)`, `LTE (0x44)`, `GTE (0x45)`.
- **Logical:** `NOT (0x46)` pops a boolean and pushes its logical negation.

Control Flow

- `JUMP (0x50)`: Consumes a 2-byte operand and unconditionally adds it to the Instruction Pointer (IP).
- `JUMP_IF_FALSE (0x51)`: Pops a boolean. If `false`, consumes a 2-byte operand and branches the IP. Otherwise, skips the operand.
- `JUMP_IF_TRUE (0x52)`: Pops a boolean. If `true`, branches the IP. Used for `||` short-circuit optimizations.

Functions & Closures

- `CALL (0x60)`: Consumes a 1-byte argument count. Pops the arguments and closure, instantiating a new Call Frame.
- `CALL_INTRINSIC (0x61)`: Consumes a 1-byte Intrinsic ID and a 1-byte argument count. Executes native C-bound functions (e.g., I/O).
- `TAIL_CALL (0x62)`: Consumes a 1-byte argument count. Overwrites the current Call Frame, preserving $O(1)$ memory during recursion.
- `RETURN (0x63)`: Pops the return value, destroys the current Call Frame, restores the parent frame, and pushes the result.
- `MAKE_CLOSURE (0x64)`: Consumes a 1-byte upvalue count. Pops a function pointer and dynamically allocates a closure object on the heap, capturing the specified surrounding variables.

Memory Allocation & Data Structures

- `ALLOC_CON (0x70)`: Consumes a 1-byte Tag ID. Pops a payload (if applicable) and allocates a tagged ADT object on the heap.
- `ALLOC_TUPLE (0x71)`: Consumes a 1-byte size operand. Pops N items and packs them into a heap-allocated tuple.

- `ALLOC_ARRAY (0x72)`: Consumes a 1-byte size operand to allocate a contiguous heap array.
- `ALLOC_ARRAY_LONG (0x7A)`: Consumes a 2-byte size operand for arrays exceeding 255 elements.
- `LIST_EMPTY (0x73)`: Pushes the `Nil` (`[]`) singleton to the stack.
- `LIST_CONS (0x74)`: Pops a Tail and a Head, allocating a linked-list node on the heap.

Pattern Matching

- `MATCH_TAG (0x80)`: Consumes a 1-byte Tag ID. Peeks at the top ADT; pushes `true` if the tags match, `false` otherwise.
- `UNPACK_CON (0x81)`: Pops an ADT and pushes its inner payload to the stack.
- `UNPACK_TUPLE (0x82)`: Pops a tuple and pushes all its constituent elements sequentially onto the stack.
- `UNPACK_LIST (0x83)`: Pops a Cons node and pushes its Head, followed by its Tail.
- `IS_LIST_EMPTY (0x84)`: Peeks at the top list node; pushes `true` if it is `Nil`, `false` otherwise.
- `MATCH_ARRAY_LENGTH (0x85)`: Pops an integer length and peeks at an array; pushes `true` if their lengths match.
- `UNPACK_ARRAY (0x86)`: Pops an array and pushes all N of its elements to the stack.
- `MATCH_FAIL (0x87)`: Triggers a fatal runtime panic indicating a non-exhaustive pattern fallthrough.

8 The Cera Virtual Machine (CeraVM)

The Cera Virtual Machine (CeraVM) is a high-performance, deterministic execution environment written entirely in C. It is engineered as a strict Stack-Machine architecture, operating over a custom Instruction Set Architecture (ISA) designed to natively execute pure functional paradigms. By decoupling the execution environment from the C# frontend compiler, CeraVM ensures cross-platform portability, deterministic memory management, and highly optimized Fetch-Decode-Execute cycles.

8.1 Architectural Overview

At its core, CeraVM bypasses traditional register-based execution in favor of an operand stack. All arithmetic, logical, and memory operations implicitly consume their operands from the top of the stack and push their resulting computations back onto it.

The architecture is broadly divided into three cooperating systems:

- **The Execution Engine:** Manages the instruction pointer (IP), decodes opcodes, and drives the continuous Fetch-Decode-Execute loop.
- **The Call Stack (Activation Records):** Maintains lexical scoping and variable bindings through dynamically allocated `CallFrame` structures.
- **The Managed Heap:** Handles dynamic memory allocations for complex data types (such as Closures, Algebraic Data Types, and Arrays) using deterministic reference counting.

8.2 Memory Management and Reference Counting

Unlike traditional imperative languages that rely on non-deterministic Mark-and-Sweep Garbage Collectors, CeraVM utilizes strict Reference Counting to manage its heap allocations. Because Cera enforces absolute functional purity and zero mutable state, it is mathematically impossible to dynamically construct cyclic memory references at runtime. This architectural guarantee enables the VM to cleanly deallocate memory with $O(1)$ temporal overhead.

- **Heap Objects:** Any structure larger than a primitive 64-bit value is allocated on the C-heap as an `Obj`. This includes `ObjString`, `ObjTuple`, `ObjArray`, `ObjList`, `ObjADT`, and `ObjClosure`. Every object begins with a standardized header tracking its `type` and `ref_count`.
- **Retain and Release:** The VM exposes `retain()` and `release()` macros directly to the opcode execution loop. When a pointer is duplicated via the `OP_DUP` instruction, or bound to a local stack slot, its reference count is incremented.
- **Deterministic Reclamation:** When an object's reference count reaches zero via a `release()` call, the VM immediately intercepts the deallocation via `freeObject()`. If the object is a composite structure (like an `ObjArray` or `ObjList`), the VM recursively releases all constituent elements before freeing the parent struct, preventing memory leaks.

8.3 The Call Stack and Lexical Scoping

CeraVM maintains a contiguous, static array of `CallFrame` structures to manage function invocations. Each frame encapsulates the execution context of a specific function or lambda.

- **Frame Structure:** A `CallFrame` holds a pointer to the executing `ObjClosure`, the current Instruction Pointer (`ip`), and a `slots` pointer. The `slots` pointer anchors the function's local variables to a specific sliding window on the global operand stack.
- **Local Variables:** When the compiler emits `OP_LOAD_LOCAL` or `OP_STORE_LOCAL`, it provides a 1-byte integer operand representing an index. The VM calculates the physical memory address by adding this index to the frame's `slots` pointer, guaranteeing $O(1)$ spatial access to scoped variables.
- **Tail Call Optimization (TCO):** To support unbounded functional recursion without triggering Stack Overflow exceptions, the VM natively intercepts the `OP_TAIL_CALL` instruction. Instead of allocating a new `CallFrame`, the VM evaluates the recursive arguments, safely tears down the current local variables via `release()`, and overwrites the active `CallFrame` window with the new arguments before resetting the Instruction Pointer. This guarantees that iterative algorithms operate with $O(1)$ stack space complexity.

8.4 Closures and Upvalues

First-class functions in CeraVM are managed via closures. A closure securely encapsulates a function pointer alongside the environmental state it captured upon creation.

- **Dynamic Capture:** When the VM executes `OP_MAKE_CLOSURE`, it dynamically allocates an `ObjClosure` on the heap. The VM then iterates over the compiler-provided arguments to capture local variables from the active stack frame into `ObjUpvalue` structures.
- **Open vs. Closed Upvalues:** While the parent function is actively executing, the upvalue remains "open," pointing directly to the live memory address on the stack. When the parent function executes an `OP_RETURN`, the VM intercepts the operation via `close_upvalues()`.

The active values are physically copied from the stack into the heap-allocated `ObjUpvalue`, securing the state for the lambda function to access long after the parent scope is destroyed.

8.5 Algebraic Data Types and Pattern Matching

CeraVM handles Algebraic Data Types (ADTs) as dynamically tagged heap structures.

- **ADT Construction:** The `OP_ALLOC_CON` instruction allocates an `ObjADT` on the heap, wrapping an internal payload (if any) and a compiler-assigned `adt_tag` (e.g., `0x04` for `Ok`, `0x05` for `Error`).
- **Structural Evaluation:** When a `switch` expression evaluates a pattern, the VM utilizes `OP_MATCH_TAG` to compare the top object's internal tag against the expected bytecode argument.
- **Masquerading:** To optimize memory allocations, CeraVM permits structural masquerading. For example, a native `ObjString` can be safely evaluated by the pattern matching engine as an `ObjList`. An empty string securely matches the `Nil` tag (`0x00`), while a populated string matches the `Cons` tag (`0x01`), allowing developers to map over strings without redundant memory duplication.
- **Destructuring:** Upon a successful pattern match, the VM utilizes explicit unpacking instructions (e.g., `OP_UNPACK_CON`, `OP_UNPACK_TUPLE`, `OP_UNPACK_LIST`) to forcefully extract the encapsulated payloads from the heap and push them sequentially onto the operand stack, securely binding them to the active scope window.

8.6 The Intrinsic Pipeline

To preserve pure functional paradigms while allowing interaction with the host operating system, CeraVM isolates all side-effects into native C-intrinsics.

- **Intrinsic Dispatch:** The `OP_CALL_INTRINSIC` instruction halts standard execution and routes control to the `execute_intrinsic()` router.
- **C-Standard Interoperability:** CeraVM seamlessly bridges managed objects with unmanaged C operations. For instance, file I/O operations intercept purely functional `char list` arrays and dynamically compile them into contiguous, null-terminated C-strings via `flatten_char_list()`. These strings are dispatched to `stdio.h` processes before being safely garbage-collected, ensuring zero memory leakage during system calls.

9 Command Line Interface and Usage

To abstract the complexity of the two-stage translation process—bridging the C# frontend compiler and the C Virtual Machine—Cera provides a unified Command Line Interface (CLI) orchestrated via shell wrapper scripts.

9.1 The Execution Pipeline

The toolchain segregates the compilation and execution phases to allow for the deterministic distribution of binary object files (`.cerabc`). The CLI exposes the following core commands to interact with this pipeline:

- `cera build <file.cera> [flags]`: Invokes the C# frontend. The compiler performs lexical analysis, semantic type-checking, and bytecode emission, outputting a serialized `.cerabc` file to the local `Out/ByteCode/` directory.

- `cera run <file.cerabc> [args]`: Invokes the CeraVM backend. The virtual machine loads the pre-compiled Intermediate Representation (IR) into memory and begins the fetch-decode-execute loop.
- `cera exec <file.cera> [args]`: A sequential orchestration command. It automatically executes a `build` followed immediately by a `run`, providing a seamless execution experience during active software development.
- `cera clear`: A workspace cleanup utility that forcefully deletes the `Out/` directory, flushing all legacy bytecode and diagnostic dumps.

9.2 Compiler and Virtual Machine Flags

The CLI exposes several diagnostic and execution flags to reveal the internal state of the compilation pipeline and runtime environment. This is highly beneficial for debugging AST generation, inspecting algorithmic efficiency, and tracing Virtual Machine execution paths.

Compiler Flags (`cerac`)

These flags are passed to the `build` command to monitor the frontend translation phases:

- `-v` (`--verbose`): Enables verbose compiler logging. for (int i = 0; i ¡ varsToPop; i++)
- `-t` (`--tokens`): Dumps the linearly segmented token stream generated by the Lexical Analyzer.
- `-a` (`--ast`): Dumps the hierarchical Abstract Syntax Tree generated by the Pratt Parser.
- `-s` (`--analyzer`): Dumps the Semantic Analyzer's global environment, detailing resolved Hindley-Milner type signatures.
- `-e` (`--emitter`): Disassembles and dumps the emitted Virtual Machine bytecode and constant pool allocations.
- `-d` (`--dump`): Enables all aforementioned diagnostic outputs simultaneously.

Virtual Machine Flags (`ceravm`)

These flags are passed to the `run` command to alter the execution environment and trace runtime behavior:

- `-d`: Enables detailed Virtual Machine execution logging (e.g., tracing instruction pointers, loaded modules, and stack states).
- `-f`: Dumps the Virtual Machine execution log to a dedicated text file.
- `-s`: Disables silent mode, forcing the VM to render internal execution logs directly to the standard output.

10 Changelog & Version History

10.1 19/06/26 - 1.0

The inital release of the Cera functional programming language.

22/06/26 - 1.1

- **Hidden keyword:** `hidden`, restricts scope to defined file (e.g. c's static). Works with named functions, top level var declarations, and types. can also use to modify import, such that secondary importers will not have access to the hidden imported file.
- **Inline keyword:** `inline`, instructs the compiler to substitute the function's body directly at the call site during the intermediate code generation phase. This optimization eliminates the runtime overhead of allocating and destroying a Call Frame on the virtual machine's operand stack, trading a larger bytecode footprint for increased algorithmic execution speed.
- **Top-Level Declarations:** The `var` keyword now supports global scope declarations. These are strictly constrained to literal values, and safely eliminate the boilerplate of zero-arity constant functions.
- **DAG Module System & Execution Speedups:** The legacy text-concatenation preprocessor has been completely eradicated. Cera now utilizes a `CompilerContext` cache to parse files into a Directed Acyclic Graph. This speeds up the compilation phase, and prioritises the entry function for situtational vm speed ups.
- **Implicit If Else blocks:** if statements that return unit, will now implicitly include `else { (); }`, simplifying I/O boilerplate.
- **Natural Pattern Unpacking:** The `var` keyword now supports full structural pattern matching on the left-hand side. This allows developers to natively destructure tuples, lists, and Algebraic Data Types directly into the local environment (e.g., `var (x, y) = tup;`).
- **New Intrinsics & Std Library:** `uptime`, `timeLocal`, to complete timing implementations. `readBin`, `writeBin`, offering binary support, and `ln` and `pow` to complete math Functionality. Standard library improved to use new hidden and inline keywords, as well as new time libary.
- **Debug Improvements:** General debugging improvements, including Opcode's being shown in dump files, greater cera bash tooling and automated build process.
- **VM & Compiler Bug Fixes:** Bug fixes, including read intrinsic using flipped constructors, local counting errors.
- **Switch Guard Clauses:** Pattern matching branches can now utilize optional `if` guard clauses (e.g., `pattern if (condition) ->`). This utilizes a safe test-then-bind virtual machine architecture to evaluate boolean constraints prior to branch commitment, safely popping local variables off the stack upon failure to prevent memory leaks.

11 Cera's Future

11.1 Excluded Features

Before considering future features for the language, we shall consider previous planned additions, that where decided against for various reasons:

- **Lazy Keyword, Force Pattern:** When the language was first being developed, the idea of a `lazy` keyword was planned, allowing for delayed evaluation thunks similar to unit thunks, but caching the results upon evaluation. This aforementioned similarity, was the primary reason for it's exclusion (see design principles), as well as adding a vast, complicated system, that can be replicated without the use of unique keywords and syntax.

- **Is Keyword, As Keyword:** Similar to C# implementation, would allow type of a generic to be checked at runtime, and converted. However this is an anti-functional pattern, can be replicated with ADTs, and would require the bytecode to not use type erasure. As such this feature does not fit within the language's principles, and will not be added.
- **Inner Named Functions:** With the implementation of the 1.1 hidden modifier, and the existing ability to create inner lambda functions, there is no longer a need for this feature, and would result in overall worse code quality.
- **Exhaustive Pattern Matching:** A check that every pattern match is exhaustive, like in OCaml, has been left out of the language as I disagree with the principal. A developer will often know there input, and it adds extra boilerplate. TLDR, write better code.

11.2 Planned Features

Now we may take an optimistic look into the future of the language:

- **Default Function Values:** Similar to C#, with `param : type = value`.
- **Better Error Messages:** Certain error messages appear vague, missing end of patterns or similar syntax will point to end of file, as well as column number being hard to track. The virtual machine suffers even worse in this case, with no details of location for runtime errors.
- **Concurrency:** As Cera is a functional language, a multithreaded model would fit the language perfectly, and shall come in a later version.
- **Record:** Simply an immutable tuple under the hood, but offers named fields for each part, allowing for syntax such as `coord.x` or `coord -> x` (to be decided).
- **FFI:** Expansion of intrinsics to allow for .so and .ddl files to be managed at runtime.